

PORTADA

Arquitectura de Software

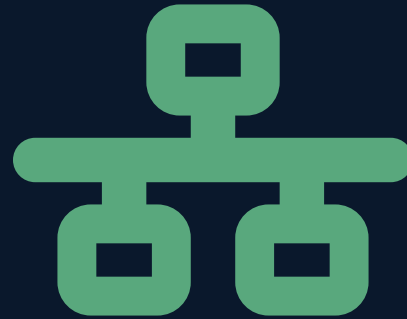
Semana 6 — Catálogo POSA: Brokers, MVC

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**Un objeto en C++
habla con otro en COBOL
sin saber dónde vive**



CORBA, 1991

Estándar	CORBA — Common Object Request Broker Architecture
Organización	OMG (Object Management Group), un consorcio de empresas tecnológicas fundado en 1989 (entre ellas 3Com, Hewlett-Packard, Sun Microsystems y Unisys)
Publicación	especificación CORBA 1.0, en 1991
Componente central	ORB — Object Request Broker
Adopción	sistemas distribuidos empresariales de los años 90 y 2000, sobre todo en telecomunicaciones, banca y sistemas militares/aeroespaciales

CORBA es la implementación histórica más citada del patrón Broker del catálogo POSA (Buschmann et al.).

CASO · POR QUÉ SE CREÓ

El problema: sistemas heterogéneos que no se entendían

En los años 80 y 90, las organizaciones grandes operaban sistemas escritos en distintos lenguajes (C, C++, COBOL, Smalltalk) sobre distintas plataformas (mainframes, Unix, Windows). Hacer que esos sistemas colaboraran significaba escribir integraciones a la medida, frágiles y costosas de mantener.

- CORBA definió un estándar independiente de lenguaje y de plataforma para invocar objetos remotos **como si fueran locales**.
- El ORB se encargaba de localizar el objeto remoto, empaquetar la petición y devolver la respuesta — el cliente no necesitaba conocer su ubicación ni su implementación.

Esa idea —ocultar dónde y cómo vive un componente— es la esencia del patrón Broker.

CASO · POR QUÉ PERDIÓ TERRENO

Un estándar poderoso, pero pesado

Pese a resolver un problema real, CORBA nunca se convirtió en la forma dominante de construir sistemas distribuidos.

- La especificación era extensa y compleja: definir interfaces requería un lenguaje propio, **IDL** (Interface Definition Language), y herramientas de generación de código específicas por lenguaje.
- Los primeros ORBs de distintos fabricantes no siempre eran interoperables entre sí, algo que contradecía la promesa original de CORBA.
- A comienzos de los 2000 surgieron alternativas más simples de adoptar: servicios web con SOAP y, más tarde, APIs REST y sistemas de mensajería — con curvas de aprendizaje mucho menores.

El patrón Broker no desapareció: sigue vivo hoy en los message brokers (RabbitMQ, Kafka) y en los API gateways, temas que veremos más adelante en el curso.

TRANSICIÓN

CORBA fue una implementación concreta de una idea más general

Detrás de CORBA hay un patrón arquitectónico documentado en el catálogo **POSA** (Pattern-Oriented Software Architecture, del libro de Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad y Michael Stal): el patrón **Broker**.

Hoy lo definimos de forma independiente de CORBA, para reconocerlo en cualquier sistema distribuido que lo use — con o sin ese nombre.

CONCEPTO

El patrón Broker

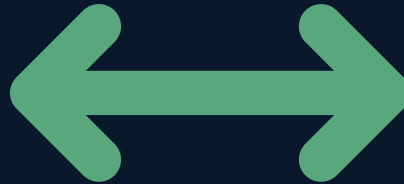
El **Broker** es un patrón arquitectónico para sistemas distribuidos: introduce un componente intermediario (el bróker) que coordina la comunicación entre clientes y servidores ubicados en procesos o máquinas distintas, ocultando su ubicación e implementación concreta.

- **Transparencia de ubicación:** el cliente invoca un servicio sin saber en qué máquina ni en qué proceso se ejecuta realmente.
- El bróker se encarga del **marshalling** y **unmarshalling** de las peticiones: convertir los datos a un formato transmisible por red, y de vuelta a su formato original al llegar a destino.

CONCEPTO · PARTICIPANTES

Quién habla con quién

- **Cliente** — invoca un servicio como si fuera una llamada local.
- **Proxy del cliente (stub)** — empaqueta la petición y la envía al bróker.
- **Bróker** — localiza el servidor adecuado y enruta la petición.
- **Proxy del servidor (skeleton)** — desempaqueta la petición para el servidor.
- **Servidor** — ejecuta la operación real y devuelve el resultado por el mismo camino.



CONCEPTO · VENTAJAS Y LIMITACIONES

El precio de la transparencia

Ventajas	Limitaciones
Desacopla cliente y servidor: cada uno puede cambiar de implementación sin afectar al otro	Añade una capa de infraestructura adicional que hay que operar y mantener
Permite ubicación transparente: los componentes pueden moverse de máquina sin romper a los clientes	Introduce latencia extra: cada llamada pasa por el bróker y por el proceso de marshalling/unmarshalling
Facilita la interoperabilidad entre lenguajes y plataformas heterogéneas	El bróker es un punto adicional que puede fallar o convertirse en cuello de botella si no se diseña bien

CONCEPTO · VIGENCIA HOY

El Broker no murió con CORBA

La idea central del patrón —un intermediario que desacopla a quien pide de quien resuelve— sigue viva en herramientas mucho más simples de adoptar que CORBA:

- **Message brokers** como RabbitMQ o Kafka: intermedian mensajes entre servicios sin que unos conozcan la ubicación de los otros.
- **API gateways**: intermedian peticiones HTTP entre clientes y microservicios, ocultando la topología interna del sistema.

Ambos temas se profundizan más adelante en el curso (BFF en la Semana 9, API Gateway en la Semana 15) — hoy solo los ubicamos como herederos conceptuales del patrón Broker.

TRANSICIÓN

Broker resuelve cómo hablan los componentes. Ahora: cómo se organiza uno solo por dentro.

El patrón Broker resuelve un problema de **comunicación entre componentes distribuidos**, en máquinas o procesos distintos.

El siguiente patrón del catálogo POSA resuelve un problema muy distinto: cómo organizar, **dentro de una sola aplicación**, la interfaz que ve el usuario.

CASO · FICHA TÉCNICA

Smalltalk-80, 1979

Origen del patrón	Modelo-Vista-Controlador (MVC)
Año	1979
Autor	Trygve Reenskaug
Institución	Xerox PARC (Palo Alto Research Center)
Entorno	Smalltalk, uno de los primeros lenguajes orientados a objetos con entorno gráfico propio
Propósito original	Organizar interfaces gráficas de escritorio (GUI) construidas en Smalltalk

MVC es, junto con Broker, uno de los patrones más antiguos y citados del catálogo POSA.

CASO · CONTEXTO

Xerox PARC y el problema de la interfaz gráfica

A finales de los 70, Xerox PARC investigaba interfaces gráficas de usuario (GUI) —ventanas, menús, íconos— años antes de que fueran comunes. Trygve Reenskaug trabajaba con el equipo de Smalltalk cuando identificó un problema recurrente en esas aplicaciones gráficas.

- Si los datos, la presentación en pantalla y el manejo de la entrada del usuario vivían mezclados en el mismo código, cualquier cambio visual arriesgaba romper la lógica de negocio.
- Reenskaug propuso separar esas tres responsabilidades en piezas independientes que colaboran entre sí — el origen del patrón MVC.

CASO · DE GUIs DE ESCRITORIO A LA WEB

Un patrón de 1979 que sigue vivo en la web

MVC nació para aplicaciones de escritorio, pero su idea de separar responsabilidades resultó igual de útil para aplicaciones web décadas después. Numerosos frameworks web adoptaron una versión adaptada del patrón:

- **Ruby on Rails** (ya visto como caso real en la Semana 2) popularizó "MVC" como término familiar para desarrolladores web.
- **Django** (Python), **Spring MVC** (Java) y **ASP.NET MVC** (.NET) adoptaron variantes del mismo patrón para organizar aplicaciones web.

Cuatro décadas después de Smalltalk-80, "MVC" sigue siendo el vocabulario común para hablar de cómo se organiza la interfaz de una aplicación.

TRANSICIÓN

De Smalltalk-80 al patrón general: qué es exactamente MVC

Igual que hicimos con Broker y CORBA, ahora separamos el patrón **MVC** de su origen histórico en Smalltalk-80, para poder reconocerlo en cualquier aplicación —de escritorio, web o móvil— que separe sus responsabilidades de la misma forma.

CONCEPTO

El patrón MVC: tres responsabilidades

MVC (Modelo-Vista-Controlador) organiza una aplicación separando tres responsabilidades independientes, para desacoplar la lógica de negocio de la interfaz que la muestra:

- **Modelo:** los datos y la lógica de negocio — no sabe nada de cómo se presenta en pantalla.
- **Vista:** la presentación visual de los datos del modelo — no contiene lógica de negocio.
- **Controlador:** recibe la entrada del usuario (clics, formularios), decide qué hacer y coordina al Modelo y a la Vista.

CONCEPTO · INTERACCIONES

Cómo colaboran las tres piezas

- El usuario actúa sobre la interfaz → la acción llega al **Controlador**.
- El Controlador interpreta la acción y actualiza el **Modelo** (por ejemplo, guarda un dato nuevo).
- El Modelo cambia de estado y lo notifica → la **Vista** se actualiza para reflejar el nuevo estado.
- La Vista nunca modifica el Modelo directamente, ni el Modelo conoce los detalles de cómo luce la Vista.



TRANSICIÓN · BROKER VS. MVC

Dos patrones, dos problemas distintos

Broker resuelve cómo se comunican componentes que viven en **máquinas o procesos distintos** — es un patrón para sistemas distribuidos.

MVC resuelve cómo se organiza el código **dentro de una sola aplicación** — es un patrón para separar responsabilidades internas, no para distribuir componentes en la red.

Confundir ambos niveles de problema es un error común — lo revisamos a continuación con una confusión todavía más frecuente.

TENSIÓN CONCEPTUAL

MVC no es lo mismo que "arquitectura de 3 capas"

Es muy común escuchar "mi aplicación usa MVC" queriendo decir "mi aplicación tiene una capa de presentación, una de lógica y una de datos". **No son lo mismo:**

- MVC describe cómo se organiza la **interfaz de usuario** en tres roles que colaboran (Modelo, Vista, Controlador) dentro de la capa de presentación.
- La arquitectura de **N capas** describe cómo se organiza el sistema completo en niveles horizontales (presentación, negocio, datos), sin importar cómo esté construida cada capa por dentro.

Un sistema de 3 capas puede usar MVC solo dentro de su capa de presentación — son decisiones en niveles distintos. Este contraste se profundiza en la sesión de Cliente/Servidor y arquitectura N capas.



INTERACCIÓN

¿Broker, MVC, o ambos?

Para cada escenario, identifiquen si el patrón que resuelve el problema es Broker, MVC, o si aplican los dos a la vez:

1. Un formulario web donde el navegador solo muestra datos y un controlador decide qué guardar.
2. Un servicio bancario que invoca, sin conocer su ubicación física, un servicio de validación de crédito alojado en otro país.
3. Una app de escritorio que consulta datos en un servidor remoto **y** organiza su propia pantalla con Modelo, Vista y Controlador.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. El patrón **Broker** introduce un intermediario que oculta la ubicación e implementación de servidores remotos, dando transparencia de ubicación a cambio de latencia y complejidad de infraestructura.
2. **CORBA** (OMG, 1991) fue la implementación histórica más influyente de Broker; perdió terreno frente a alternativas más simples, pero su idea sigue viva en message brokers y API gateways.
3. El patrón **MVC** separa Modelo, Vista y Controlador para desacoplar la lógica de negocio de la interfaz de usuario.
4. **Smalltalk-80** (Trygve Reenskaug, Xerox PARC, 1979) es el origen documentado de MVC; el patrón migró de las GUIs de escritorio a frameworks web como Rails, Django, Spring MVC y ASP.NET MVC.
5. Broker resuelve **comunicación entre componentes distribuidos**; MVC resuelve **organización interna de una aplicación** — no son intercambiables, y MVC tampoco equivale a una arquitectura de N capas.

CIERRE

Para la próxima semana

Seguimos recorriendo el catálogo POSA. La próxima sesión cubre dos patrones más:

- **Batch:** procesamiento de grandes volúmenes de datos sin intervención del usuario, por lotes.
- **Cliente/Servidor:** el patrón base de casi toda la Unidad 2 — y donde retomamos la distinción entre MVC y la arquitectura de N capas que dejamos abierta hoy.

